다차원 배열

다차원 배열

- 2차원 배열의 선언과 초기화

```
int[][] intArray;
int[] intArray[];
int intArray[][];
```

2차원 배열의 선언과 초기화를 동시에

1

선언과 동시에 초기화 (배열의 개수는 자동으로 [2][3]이 된다)

```
int[ ][ ] ar = new int[ ][ ]{{100, 200, 300}, {400,500,600}};
```

클래스명

2

선언과 동시에 초기화 (배열의 개수는 자동으로 [2][3]이 된다)

```
int[ ][ ] ar = {{100, 200, 300}, {400,500,600}};
```

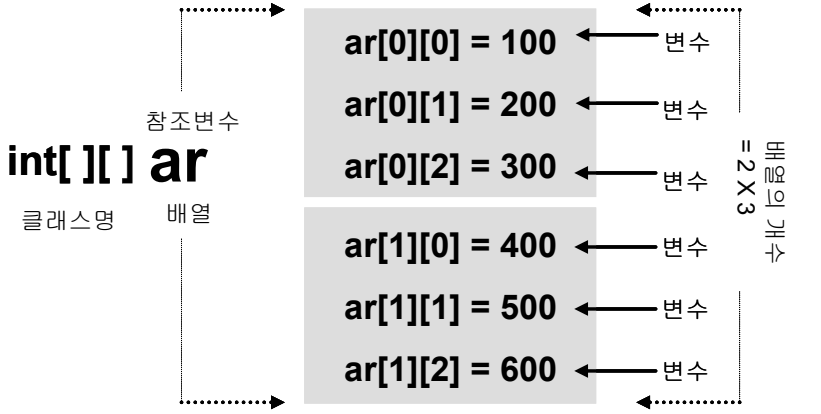
클래스명

이 방법들은 선언과 초기화를 동시에 하는 방법입니다. 위의 방법 둘 다 사용할 수 있습니다.

2차원 배열의 선언과 할당의 분리

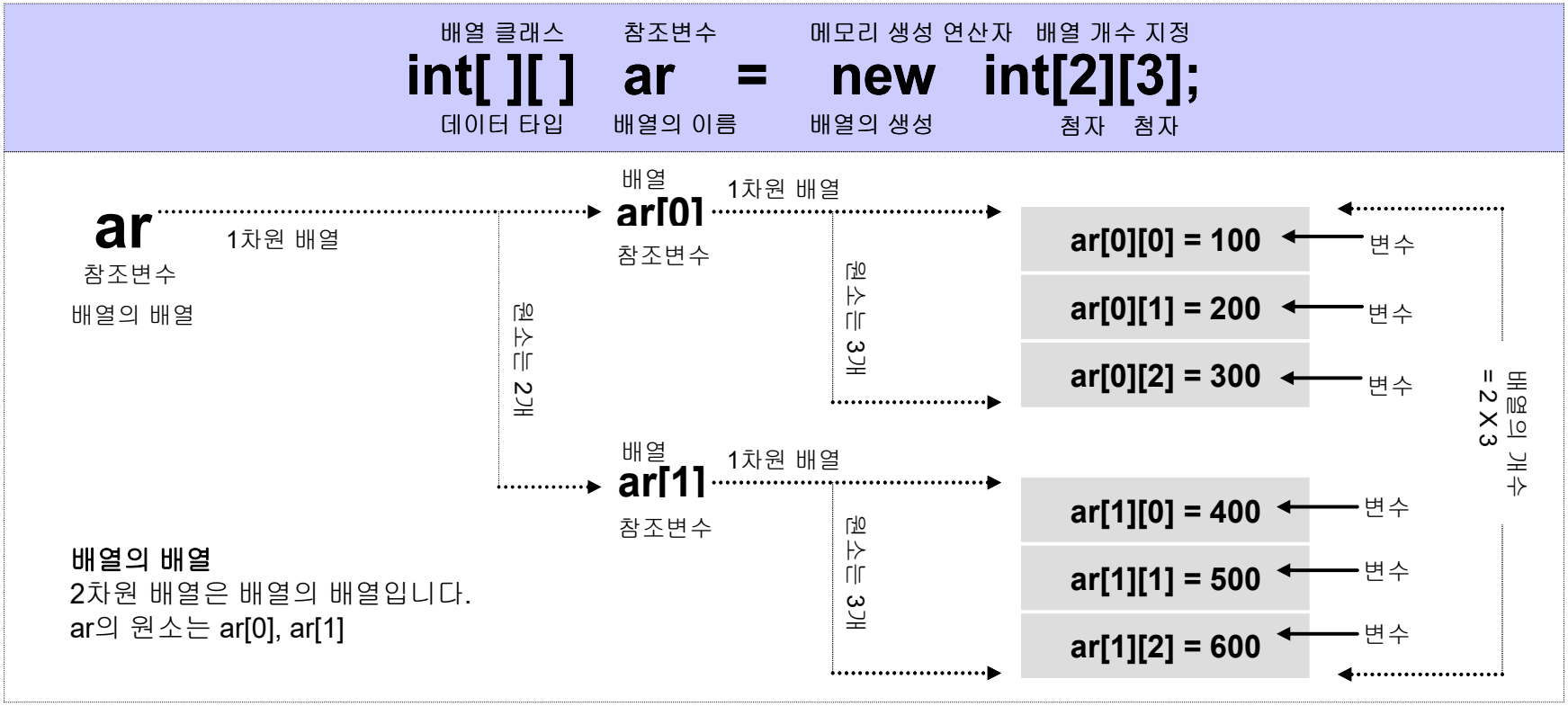
배열 클래스 참조변수 메모리 생성 연산자 배열 개수 지정

```
int[ ][ ] ar = new int[2][3];
```



다차원 배열

2차원 배열의 분석(배열의 배열)



- ar.length : 2
- ar[0].length : 3
- ar[1].length : 3

실습 예제. 성적 총점 구하기

- 문제 : 4명의 학생(홍길동, 고길동, 김길동, 이길동)의 국어, 영어,수학 시험점수가 있고, 이 학생들의 총점을 구하시오.

  - 홍길동 >> 10 20 30 : 60
  - 고길동 >> 20 30 40 : 90
  - 김길동 >> 10 25 30 : 65
  - 이길동 >> 30 30 40 : 100

가변 배열

- 2차원 이상의 다차원 배열을 생성할 때 전체 배열 차수 중 마지막 차수의 길이를 지정하지 않고 생성한 배열

```
int[][] score = new int [3][];
score[0] = new int[4];
score[1] = new int[2];
score[2] = new int[6];
```

실습 예제. 파일로 부터 배열 생성하기

- 문제 : 파일로 부터 학생 데이터를 읽어, 2차원 배열을 초기화 한 후, 총점을 구한 후 출력한다.

```
File file = new File( pathname: "res/scores.txt");
Scanner scan = new Scanner(file);
```

- <입출력 예시>

scores.txt

```
4
3 홍길동 10 20 30
4 고길동 20 30 40 50
3 김길동 10 25 30
2 이길동 30 40
```

```
홍길동 >> 10 20 30 : 60
고길동 >> 20 30 40 50 : 140
김길동 >> 10 25 30 : 65
이길동 >> 30 40 : 70
```

객체 지향 프로그래밍

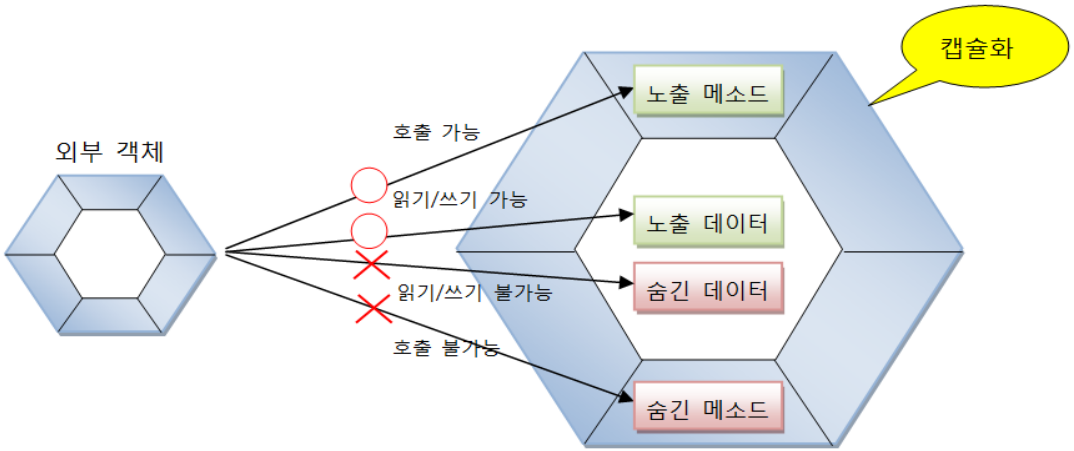
객체지향 프로그래밍의 특징

- 객체지향 프로그래밍(Object Oriented Programming)
  - 소프트웨어를 분석, 설계하고 구현하는 많은 방법 중의 한가지 방법
  - 특정 언어와는 관련이 없음
    - OOP를 지원하는 프로그래밍 언어를 사용할 때 OOP 기법을 구현하기 용이
  - 대표적인 4가지 개념
    - **추상화(Abstraction), 은닉화/캡슐화(Encapsulation), 상속(Inheritance), 다형성(Polymorphism)**

- **추상화(Abstraction)**
  - 자세한 사항은 무시하고 본질적인 특징에 집중하도록 하는 과정
    - 불필요한 세부 사항들은 제거하고 관심있는 데이터나 기능을 추출하여 표현하는 것
      - 객체가 가져야 하는 필드와 메소드를 추출하는 행위

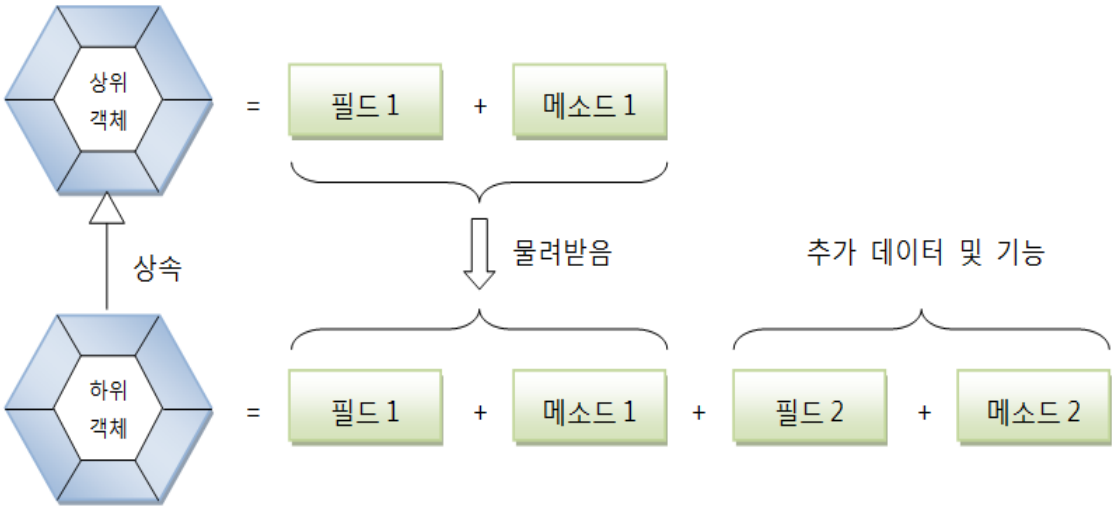
객체지향 프로그래밍의 특징

- **은닉화/캡슐화 (Encapsulation)**
  - 객체의 필드, 메소드를 하나로 묶고, 실제 구현 내용을 감추는 것 - 정보은닉
  - 외부 객체는 객체 내부 구조를 알지 못하며 객체가 노출해 제공하는 필드와 메소드만 이용 가능
  - **필드와 메소드를 캡슐화하여 보호하는 이유는 외부의 잘못된 사용으로 인해 객체가 손상되지 않도록 하기 위함**
  - **클래스를 통해 캡슐화하고, 캡슐화된 멤버를 노출시킬 것인지 숨길 것인지 결정하기 위해 접근 제한자(Access Modifier) 사용**



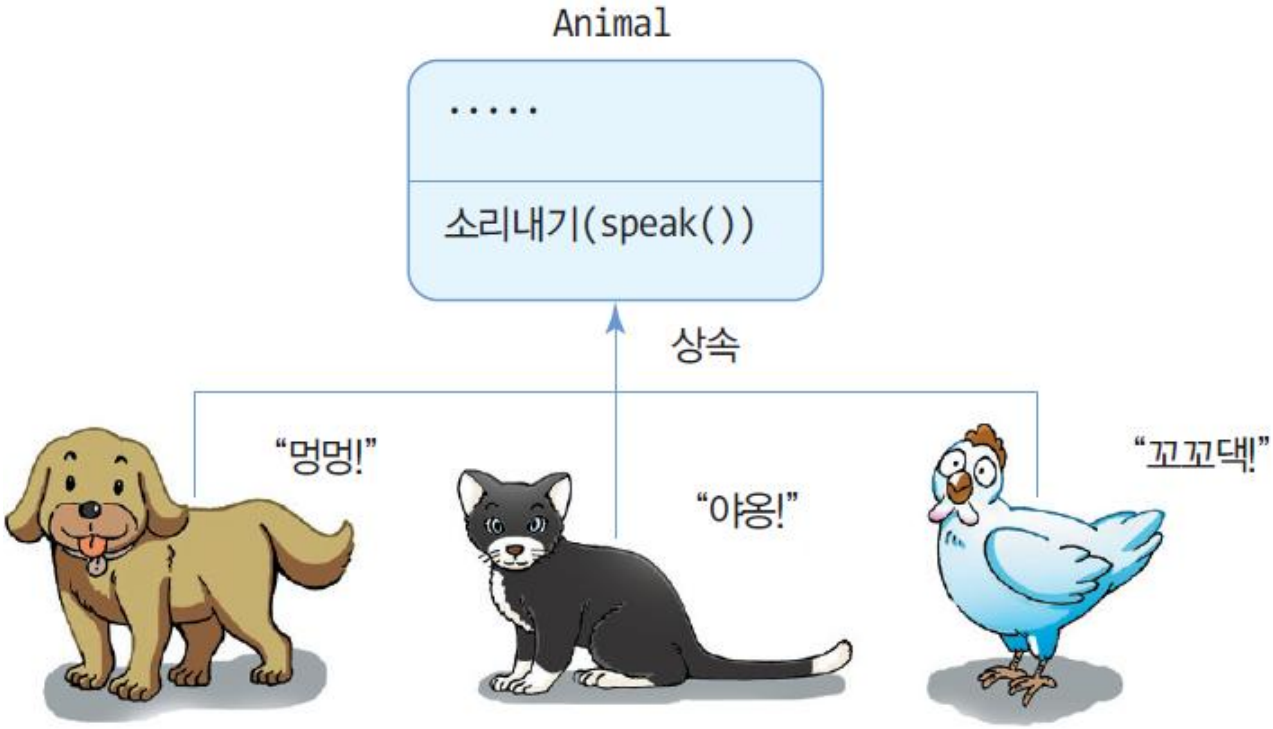
객체지향 프로그래밍의 특징

- **상속 (Inheritance)**
  - 상위(부모) 객체의 필드와 메소드를 하위(자식) 객체에게 물려주는 행위
  - 하위 객체는 상위 객체를 확장해서 추가적인 필드와 메소드를 가질 수 있음
  - **상속 대상: 필드와 메소드**
  - 상속의 효과
    - 상위 객체를 재사용해서 하위 객체를 빨리 개발 가능
    - 반복된 코드의 중복을 줄임
    - 유지 보수의 편리성 제공
    - **객체의 다형성 구현**



객체지향 프로그래밍의 특징

- **다형성 (Polymorphism)**
  - 같은 이름의 메소드가 객체에 따라 다르게 구현되는 것
    - 실행될 메소드가 코드에서 연결한 객체에 맞게 동적으로 바인딩 됨



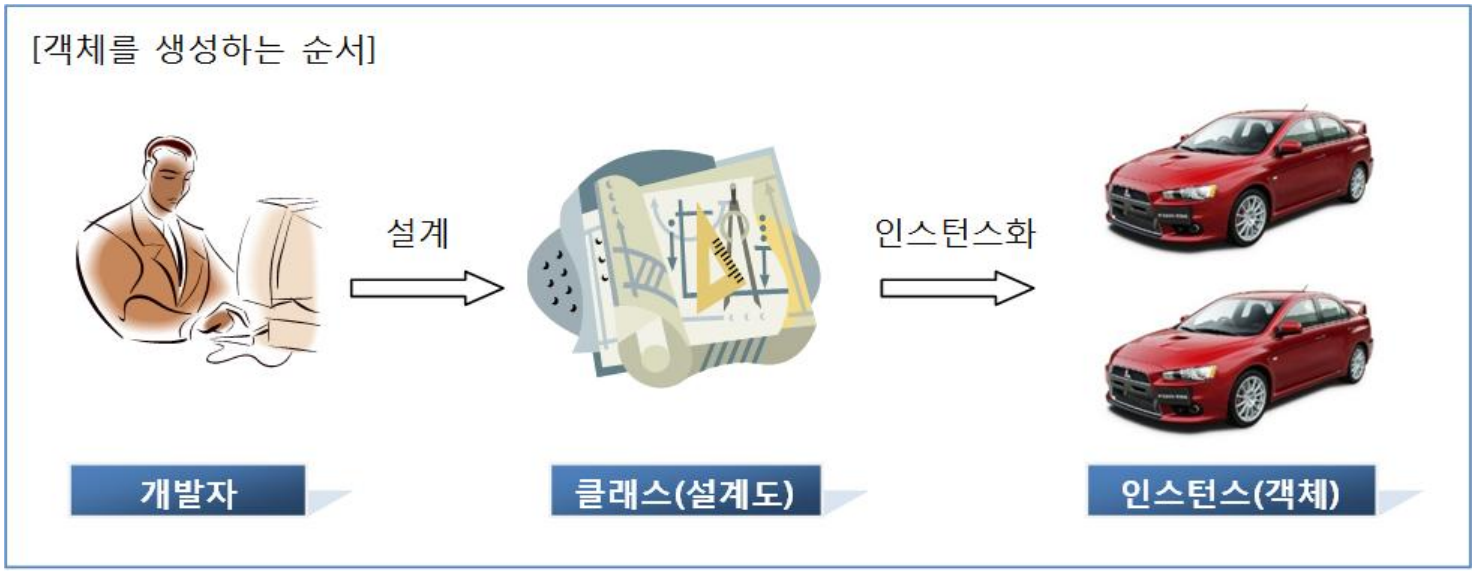
객체 지향 프로그래밍

- 객체 지향 프로그래밍 (Object Oriented Programming)
  - 부품 객체를 먼저 만들고 이것들을 하나씩 조립해 완성된 프로그램을 만드는 기법
- 객체(Object) = 데이터(필드) + 기능(메소드)
  - 물리적으로 존재하는 것
    - 자동차, 자전거, 책, 사람
  - 추상적으로 생각할 수 있는 것
    - 자신의 속성을 가지고 있어 다른 것과 식별 가능한 것 – 학과, 강의, 주문
  - 객체는 필드와 메소드로 구성된 **자바 객체로 모델링 가능**
- **자바의 객체는 클래스로부터 생성할 수 있음**

클래스

클래스와 객체

- 클래스
  - 객체의 설계도 혹은 틀
  - 객체를 생성하기 위한 필드와 메소드가 정의
- 객체
  - 클래스로 찍어낸 실체
    - 메모리 공간을 갖는 구체적인 실체, 인스턴스(instance)라고도 부름
- 하나의 클래스로부터 여러 개의 객체 생성 가능



클래스 선언

- 클래스 이름
  - 자바 식별자 작성 규칙에 따라야 함

번호	작성 규칙	예
1	하나 이상의 문자로 이루어져야 한다.	Car, SportsCar
2	첫 번째 글자는 숫자가 올 수 없다.	Car, 3Car(x)
3	'$', '_' 외의 특수 문자는 사용할 수 없다.	$Car, _Car, @Car(x), #Car(x)
4	자바 키워드는 사용할 수 없다.	int(x), for(x)

  - 첫 글자는 대문자로 시작
  - 첫 글자와 연결된 다른 단어의 첫글자는 대문자로 작성하는 것이 관례

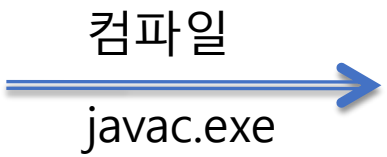
Calculator, Car, Member, ChatClient, ChatServer, Web_Browser

클래스 선언

- 소스파일 생성 : 클래스이름.java
- 소스 작성

```
public class 클래스이름 {


}
```

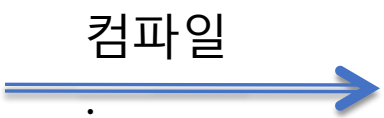


클래스이름.class

- **소스 파일당 하나의 클래스를 선언하는 것이 관례**
  - 두 개 이상의 클래스 선언도 가능하지만, 하나씩 만들자
  - 소스 파일 이름과 동일한 클래스만 public으로 선언 가능
  - 선언한 개수만큼 바이트 코드 파일이 생성

Car.java

```
public class Car {
}
class Tire {
}
```



Car.class

Tire.class

객체 생성과 참조 변수

- new 연산자
  - 객체 생성시키는 연산자
  - 힙 메모리에 객체 생성 후 객체 생성 번지 리턴

```
new 클래스();
```





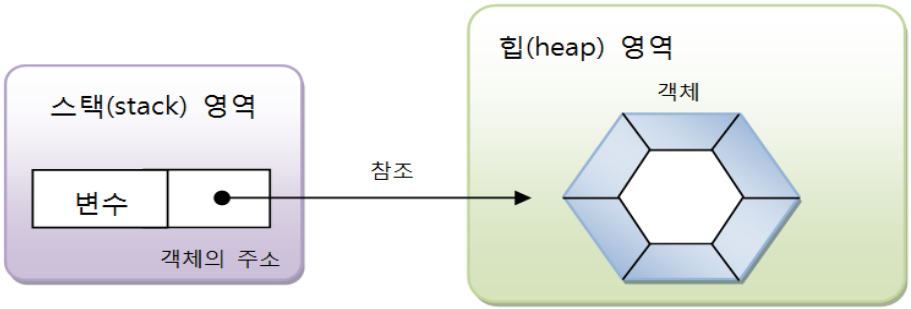
* 클래스() : 생성자를 호출하는 코드

Car myCar = new Car();

- 참조 변수 (Reference Variable)
  - new 연산자에 의해 리턴 된 객체의 번지 저장 (참조 타입 변수)
  - 힙 영역의 객체를 사용하기 위해 사용

```
클래스 변수;
변수 = new 클래스();
```

```
클래스 변수 = new 클래스();
```



클래스의 용도

- 라이브러리(API : Application Program Interface) 용
  - 자체적으로 실행되지 않음
  - 다른 클래스에서 이용할 목적으로 만든 클래스

- 실행용
  - main() 메소드를 가지고 있는 클래스로 실행할 목적으로 만든 클래스

1개의 애플리케이션 = (1개의 실행클래스) + (n개의 라이브러리 클래스)

클래스의 구성 멤버

- 필드(Field) or 멤버 변수
- 생성자(Constructor)
- 메소드(Method) or 멤버 메소드 or 멤버 함수

- **필드(Field)**
  객체의 데이터가 저장되는 곳
- **생성자(Constructor)**
  객체 생성시 초기화 역할 담당
- **메소드(Method)**
  객체의 동작에 해당하는 실행 블록

```
public class ClassName {

    //필드
    int fieldName;

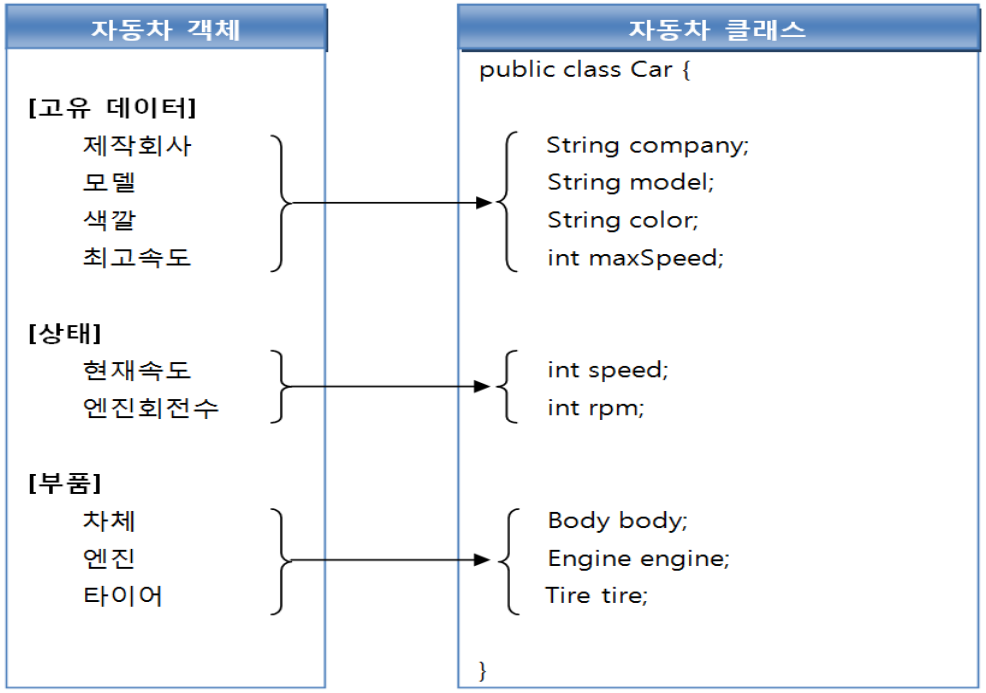
    //생성자
    ClassName() { ... }

    //메소드
    void methodName() { ... }

}
```

필드(Field)

- 필드의 내용
  - 객체의 고유 데이터
  - 객체의 현재 상태
  - 객체가 가져야 할 부품 객체

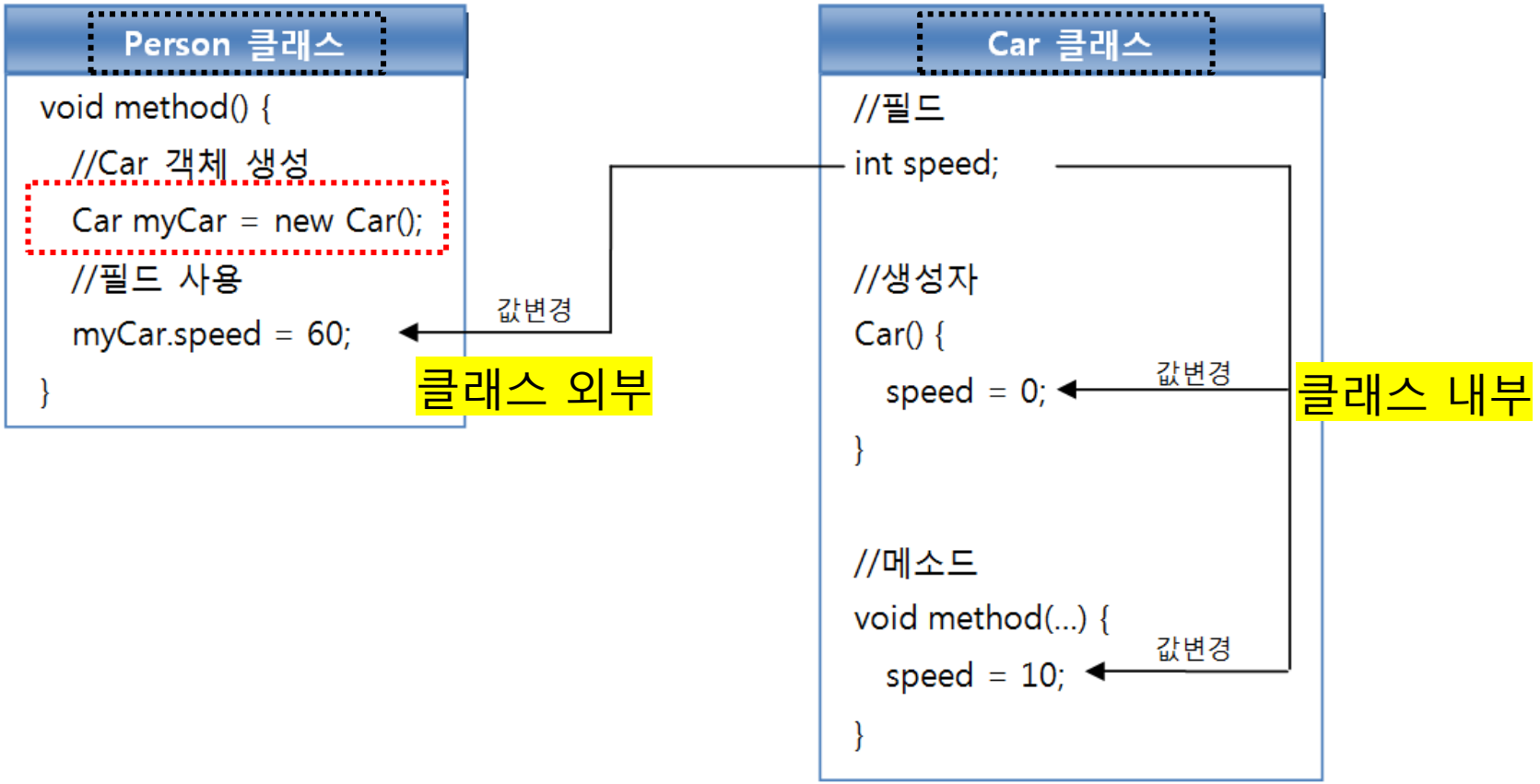


- 필드의 기본 초기값
  - 객체 생성시 자동으로 기본값으로 초기화됨

분류		데이터 타입	초기값
기본 타입	정수 타입	byte	0
		char	₩u0000 (빈 공백)
		short	0
		int	0
		long	0L
	실수 타입	float	0.0F
		double	0.0
	논리 타입	boolean	false
참조 타입		배열	null
		클래스(String 포함)	null
		인터페이스	null

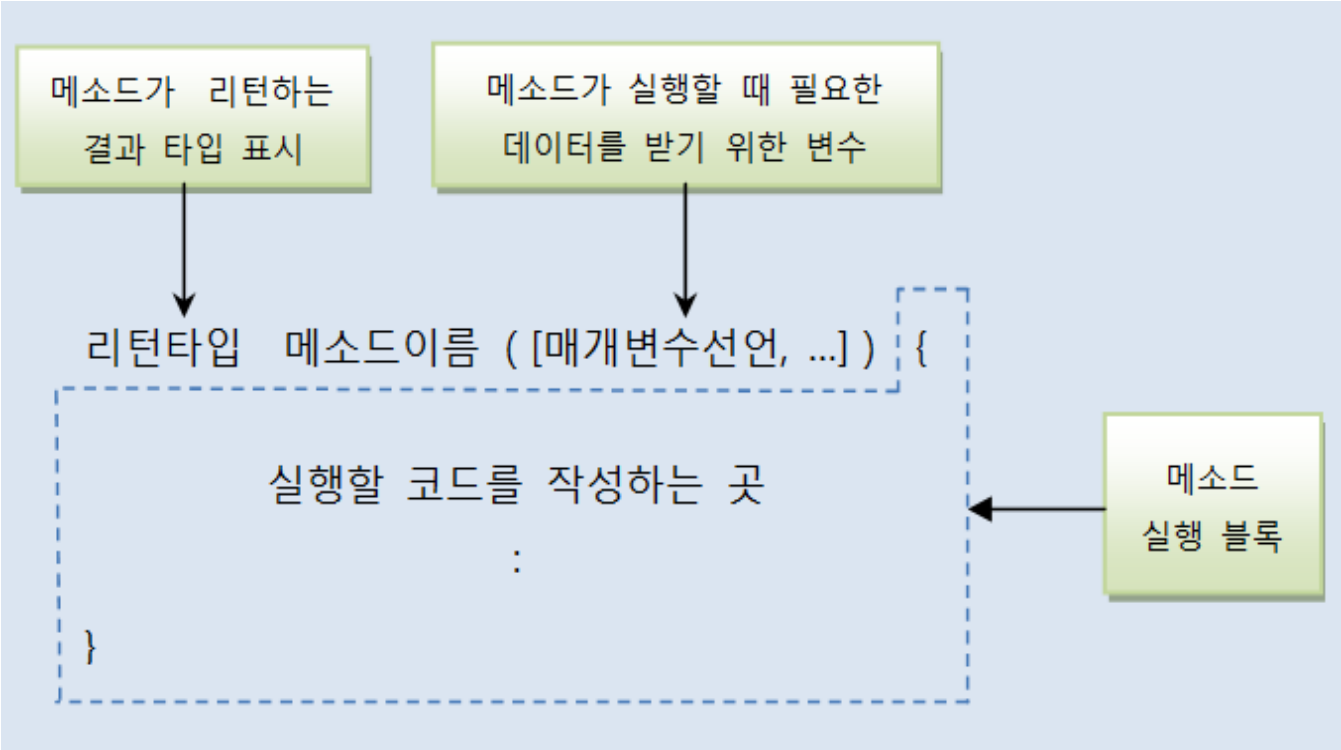
필드(Field) 사용

- 필드 사용 위치
  - 클래스 내부 : "필드이름"으로 바로 접근
  - 클래스 외부 : "변수.필드이름"으로 접근



메소드(method)

- 자바에서는 함수를 메소드라는 용어로 대체
- 메소드의 매력
  - 메소드는 작업을 하나로 묶어서 관리할 수 있다는 측면에서 효율적
  - 언제든 필요하면 반복적으로 호출 가능
  - 호출할 때 매개변수만 만족시켜 주면 언제든지 호출 가능
- 메소드 선언



Return 문

- 메소드가 정상적으로 종료되는 경우
  - 메소드의 블록{} 내에 있는 모든 문장들을 수행했을 때
  - 메소드의 블록{} 내에 있는 문장을 수행 중 return 문을 만났을 때
- return 문
  - 메소드 실행을 중지하고 리턴값을 지정하는 역할
    - 리턴값이 없는 경우, 메소드 실행을 강제 종료 시키는 역할

* 리턴 값이 없는 경우

```
return;
```

* 리턴 값이 있는 경우

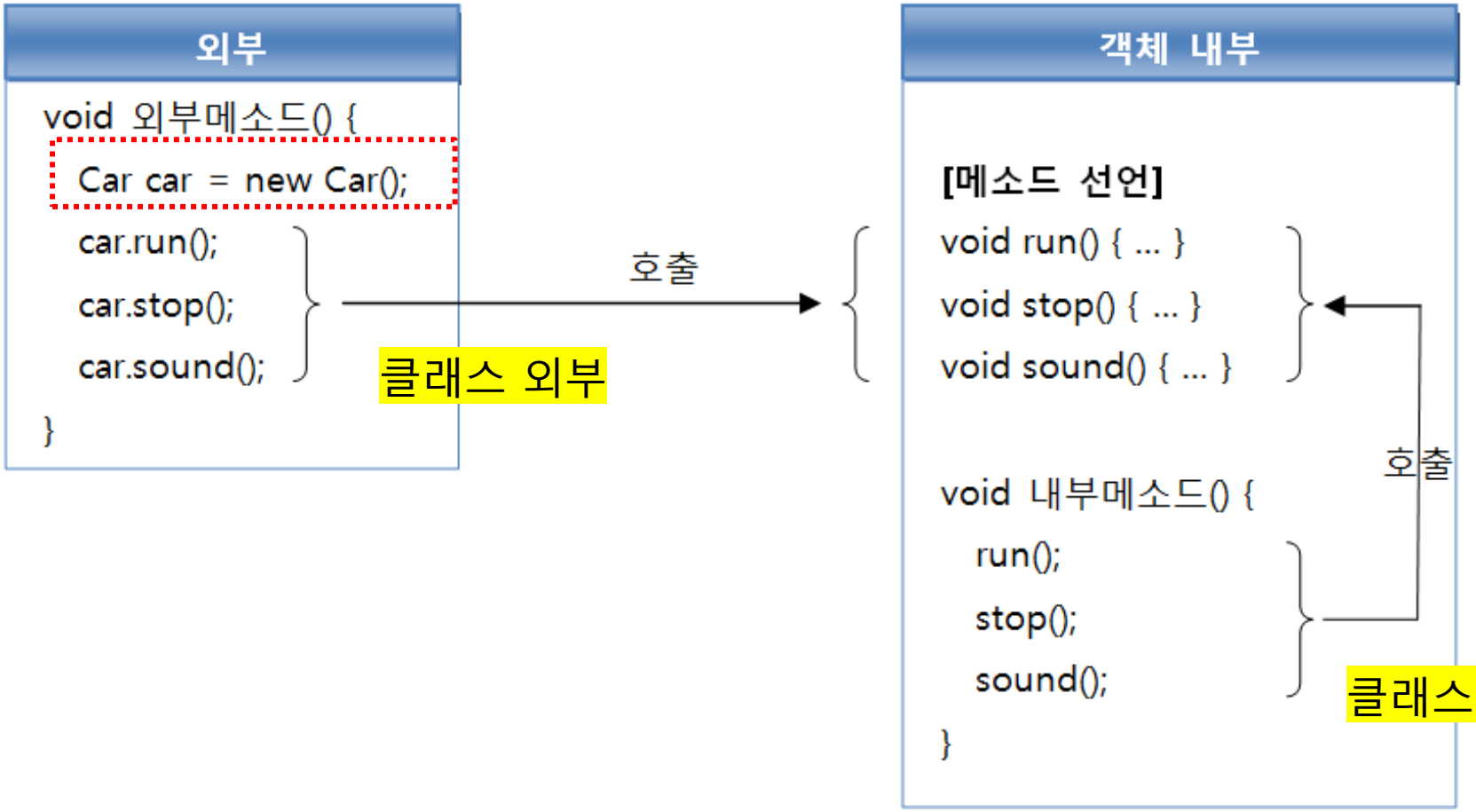
```
return 리턴값;
```

```
void powerOnOff()
{
    power = !power;
}
```

```
int add(int a, int b)
{
    int result = a + b;
    return result;
}
```

메소드 호출

- 클래스 내부 : 메소드 이름으로 호출
- 클래스 외부 : 객체 생성 후 참조 변수를 이용해 호출



예제 4-1. Circle 클래스의 객체 생성 및 활용

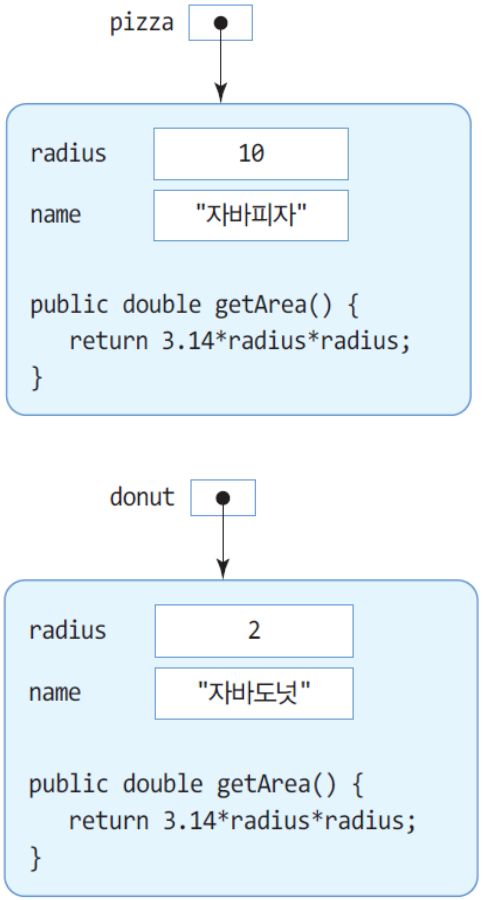
```
public class Circle {
   int radius;                  // 원의 반지름을 저장하는 멤버 변수
   String name;                 // 원의 이름을 저장하는 멤버 변수

   public double getArea() {    // 멤버 메소드
      return 3.14*radius*radius;
   }

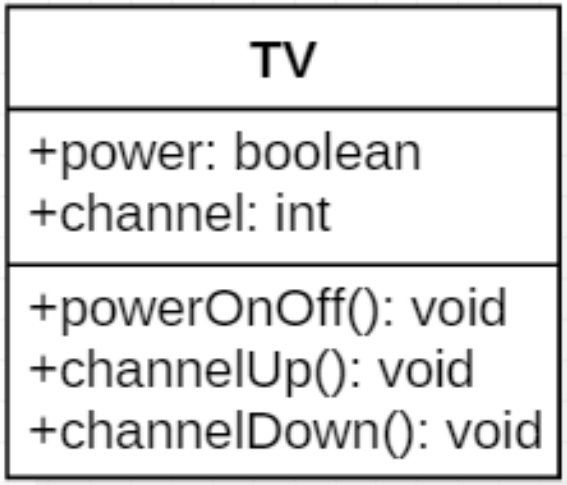
   public static void main(String[] args) {
      Circle pizza;
      pizza = new Circle();              // Circle 객체 생성
      pizza.radius = 10;                 // 피자의 반지름을 10으로 설정
      pizza.name = "자바피자";           // 피자의 이름 설정
      double area = pizza.getArea();     // 피자의 면적 알아내기
      System.out.println(pizza.name + "의 면적은 " + area);

      Circle donut = new Circle();       // Circle 객체 생성
      donut.radius = 2;                  // 도넛의 반지름을 2로 설정
      donut.name = "자바도넛";           // 도넛의 이름 설정
      area = donut.getArea();            // 도넛의 면적 알아내기
      System.out.println(donut.name + "의 면적은 " + area);
   }
}
```

```
자바피자의 면적은 314.0
자바도넛의 면적은 12.56
```



예제 4-2. TV 클래스




```
public class TV{
 public boolean power;
 public int channel;

 public void powerOnOff() {
   power = !power;
 }

 public void channelUp() {
   ++channel;
 }

 public void channelDown() {
   --channel;
 }
}
```

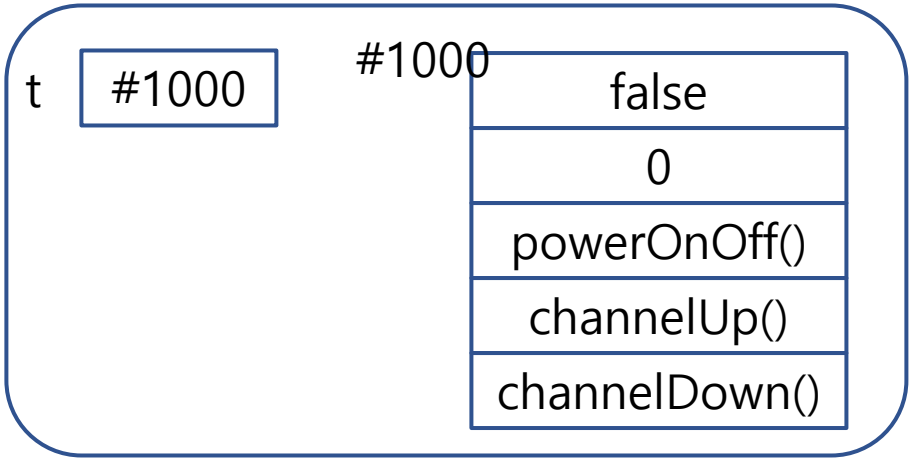
TV 클래스 : 객체의 생성과 사용

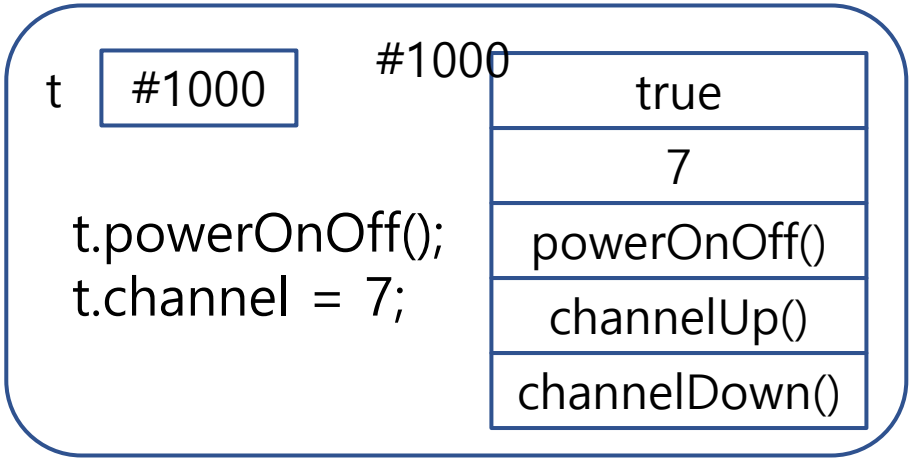
- 클래스로부터 객체 만들기 (인스턴스)

클래스명 참조변수명;

TV t;
t = new TV ();

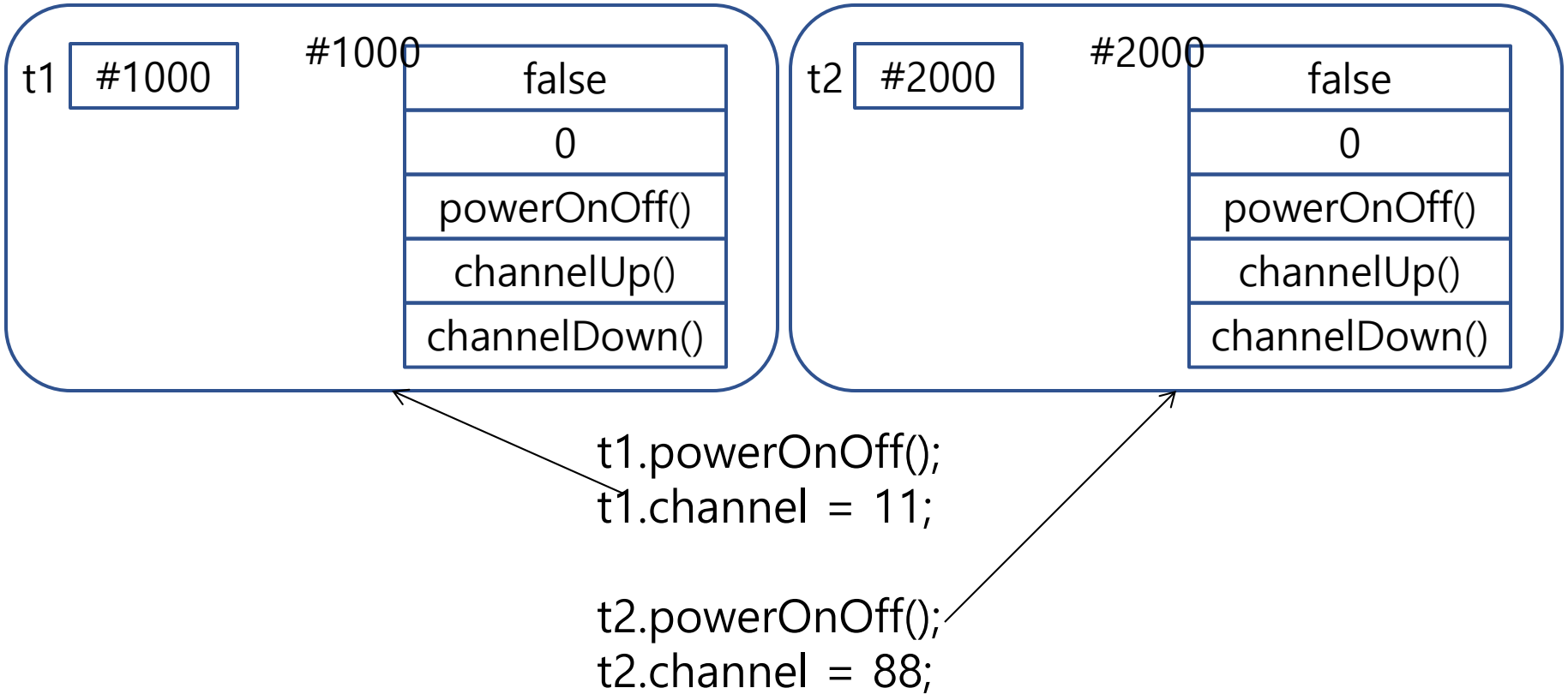
TV t = new TV();



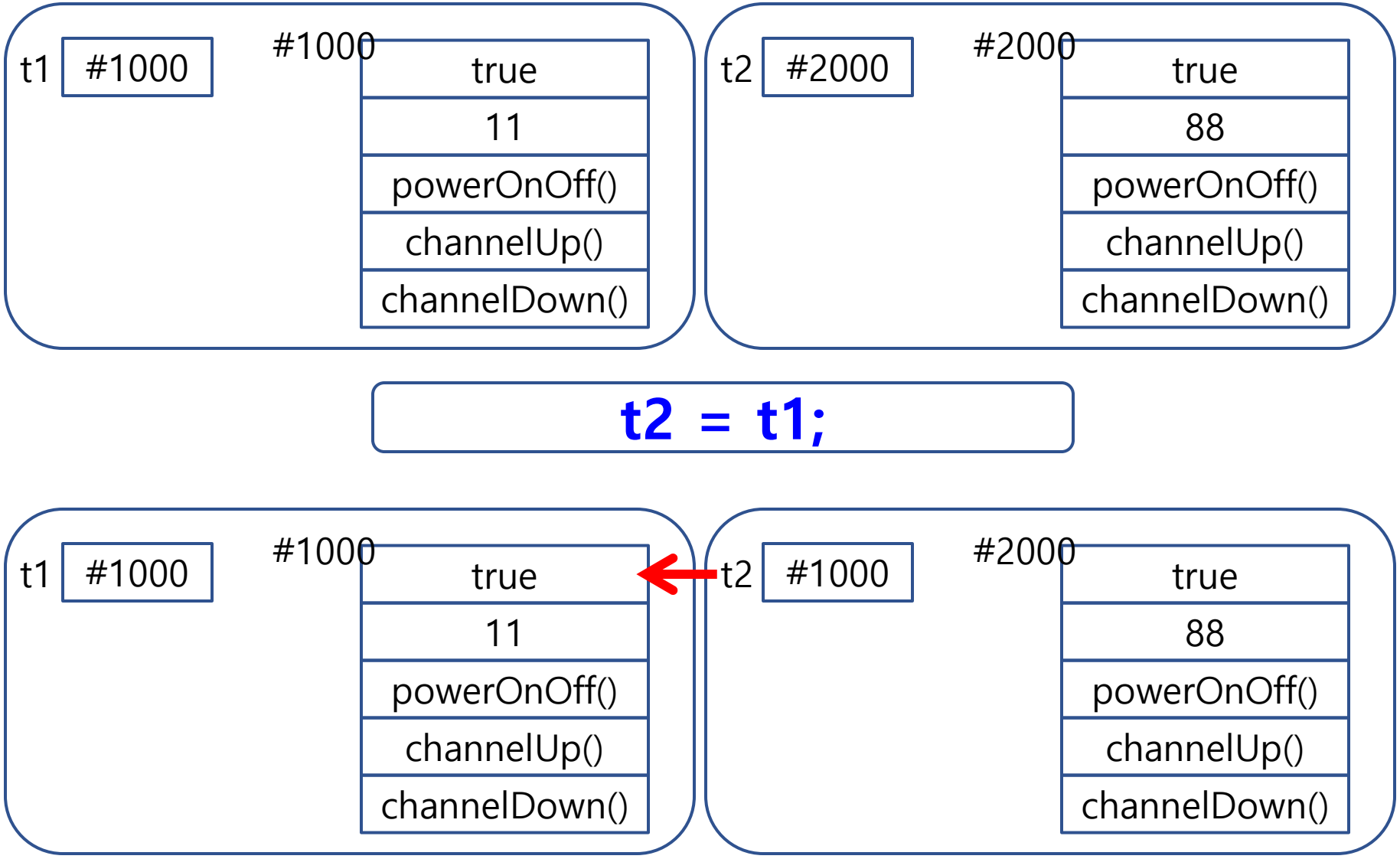


TV 클래스 : 객체의 생성과 사용

```
TV t1 = new TV();
TV t2 = new TV();
```

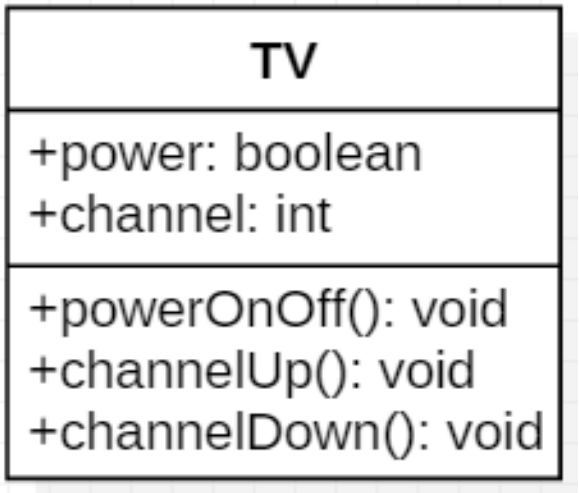


TV 클래스 : 객체의 생성과 사용



실습 예제. TV 클래스

- 채널은 0~5 사이의 값을 가질 수 있으며, 0보다 작은 경우 5, 5보다 큰 경우 0번 채널값을 갖도록 변경하세요.



확인 방법

main()에서 channelUp()을 7번, 이어서 channelDown()을 3번 호출하고 매번 channel 값을 출력하세요.

출력예시) 1 2 3 4 5 0 1 / 0 5 4

```
public class TV{
 public boolean power;
 public int channel;

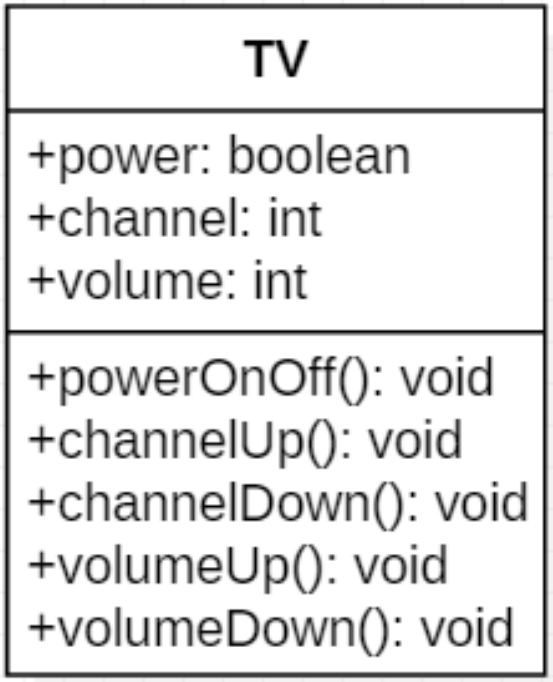
 public void powerOnOff() {
   power = !power;
 }

 public void channelUp() {
   ++channel;
 }

 public void channelDown() {
   --channel;
 }
}
```

Lab 실습. TV에 볼륨 속성 및 메소드 추가

- 앞의 TV 클래스에 볼륨 필드 및 볼륨 변경 메소드를 추가하고, 볼륨 조절 가능함을 보여주세요.
  - 단, 0<= volume <=5 사이의 값만을 가질 수 있음
    - 0보다 작으면 0, 5보다 크면 5으로 설정



제출 항목
1. 볼륨 필드와 조절 메소드가 추가된 TV 클래스
2. 경계값을 확인하는 main() 포함 실행클래스
   - 볼륨을 7번 올려도 5에서 멈추는지
   - 0에서 내려도 음수가 되지 않는지

3. 실행 결과 캡처 사진 1장

09월29일까지 이캠퍼스 제출
-. TV.java
-. TestMain.java
-. 학번이름 포함된 실행결과 캡처 화면

매개변수 전달 방식

- 기본 타입의 값이 전달 되는 경우 (**Call by Value**)
  - 호출자가 건네는 값이 매개 변수에 복사되어 전달.
  - 실인자 값은 변경되지 않음

int m = n;

```
public class CallByValue {
   public static void main(String args[]) {
      int n = 10;

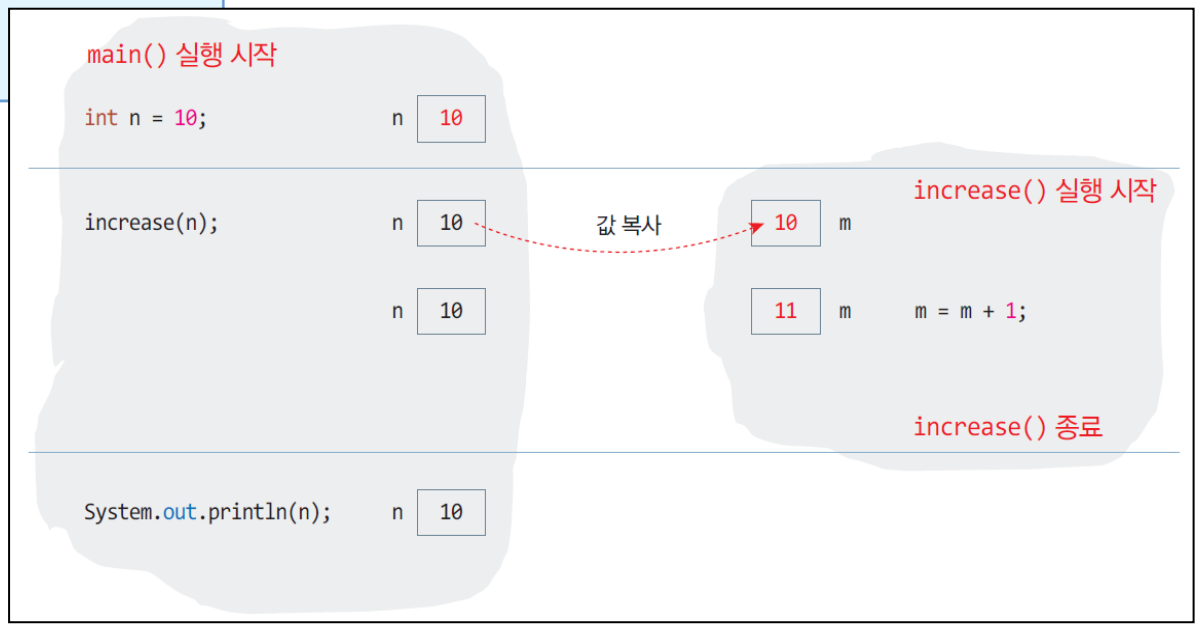
      increase(n);

      System.out.println(n);
   }
```

호출

```
   static void increase(int m) {
      m = m + 1;
   }

}
```



매개변수 전달 방식

- **객체가 전달되는 경우 (Call by Reference)**
  - 객체의 레퍼런스만 전달되며, 매개변수가 실인자 객체를 공유

Circle m = pizza;

```
public class ReferencePassing {
   public static void main (String args[]) {
      Circle pizza = new Circle(10);

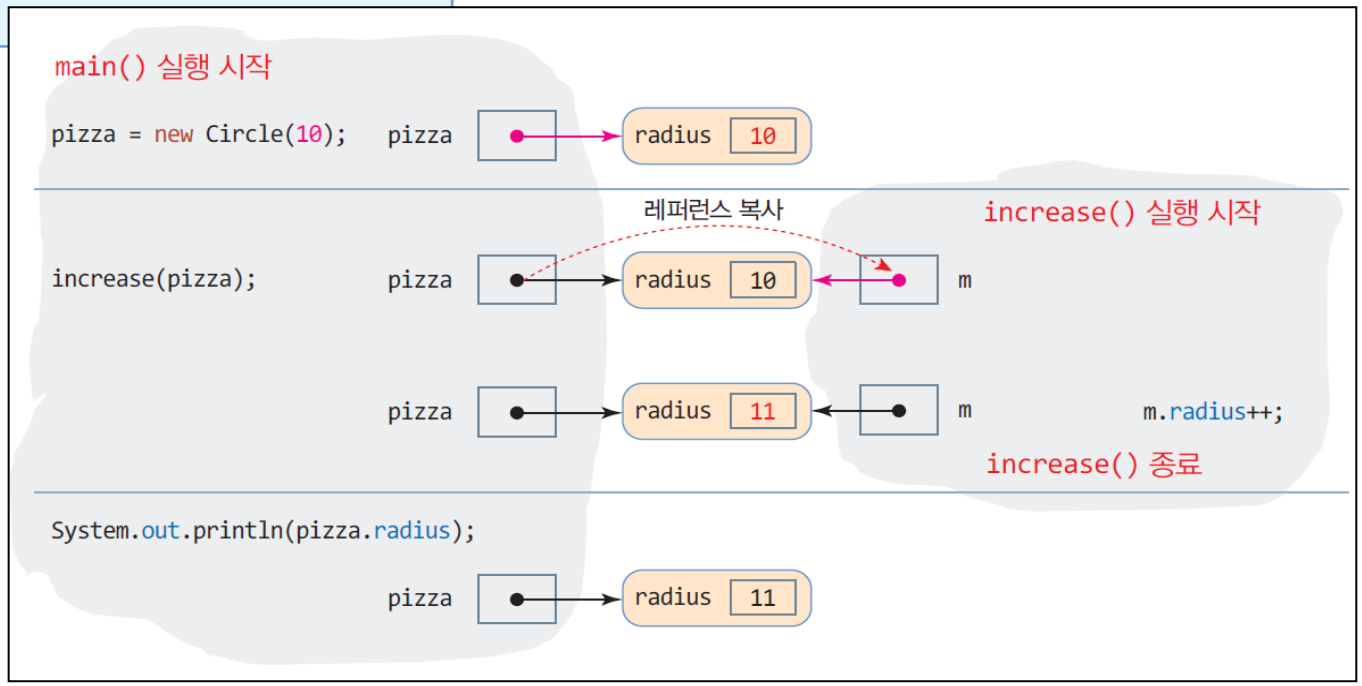
      increase(pizza);

      System.out.println(pizza.radius);
   }
```

호출

```
   static void increase(Circle m) {
      m.radius++;
   }

}
```



매개변수 전달 방식

- **매개 변수의 수를 모를 경우 (가변인수)**
  - 메소드 호출시 넘겨준 값의 수에 따라 자동으로 배열 생성

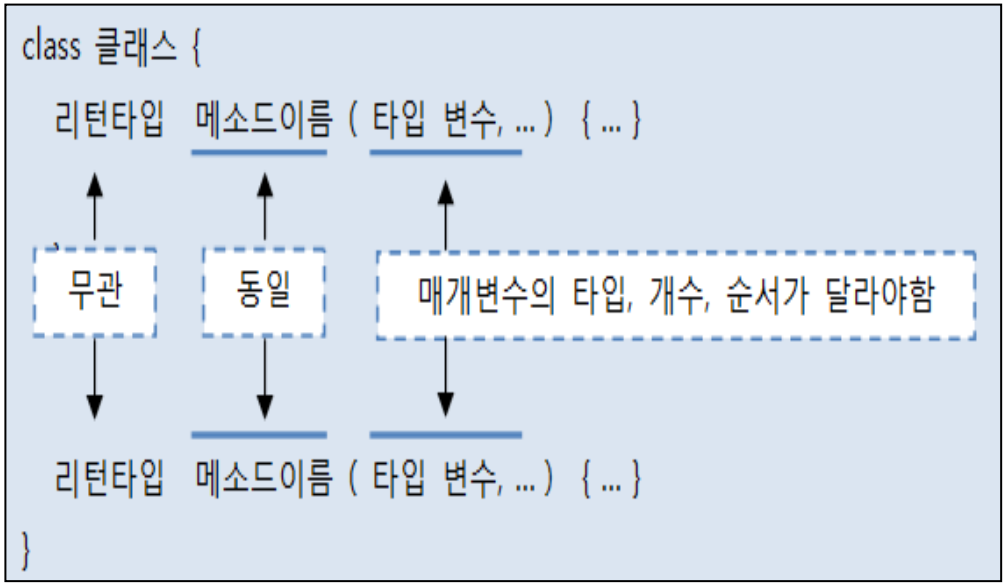
```
int sum2(int ··· values) {  }
```

```
int result = sum2(1, 2, 3);
int result = sum2(1, 2, 3, 4, 5);
```

```
int[] values = { 1, 2, 3 };
int result = sum2(values);
int result = sum2(new int[] { 1, 2, 3, 4, 5 });
```

메소드 오버로딩

- 오버로딩(Overloading)
  - 한 클래스 내에서 두 개 이상의 이름이 같은 메소드 작성
  - 매개값을 다양하게 받아 처리할 수 있도록 하기 위함
    - 메소드 이름이 동일하여야 함
    - 매개 변수의 타입, 개수, 순서가 서로 달라야 함
    - 리턴 타입은 오버로딩과 관련 없음



System.out.println() 메소드

```
void println() { .. }
void println(boolean x) { .. }
void println(char x) { .. }
void println(char[] x) { .. }
void println(double x) { .. }
void println(float x) { .. }
void println(int x) { .. }
void println(long x) { .. }
void println(Object x) { .. }
void println(String x) { .. }
```

메소드 오버로딩

- 오버로딩된 메소드 호출하는 경우 JVM은 매개값을 보고 메소드를 선택

```
public static void main(String args[]) {
    MethodSample a = new MethodSample();

    int i = a.getSum(1, 2);

    int j = a.getSum(1, 2, 3);

    double k = a.getSum(1.1, 2.2);

}
```

매개 변수의 개수와 타입이 서로 다른 3 함수 호출

```
public class MethodSample {
    public int getSum(int i, int j) {
        return i + j;
    }

    public int getSum(int i, int j, int k) {
        return i + j + k;
    }

    public double getSum(double i, double j) {
        return i + j;
    }
}
```

오류

```
class MethodOverloadingFail { // 메소드 오버로딩이 실패한 사례
    public int getSum(int i, int j) {
        return i + j;
    }

    public double getSum(int i, int j) {
        return (double)(i + j);
    }
}
```

요약

- 객체
  - 데이터(필드) + 기능(메소드)의 덩어리데이터
- 클래스
  - 객체의 설계도
- 객체 생성
  - new 가 힙에 객체를 만들고 그 번지를 돌려줌
- 참조 변수
  - 객체가 아니라 객체의 번지를 담는 변수
- 필드 기본값
  - 정수 0, 실수 0.0, boolean false, 참조 타입 null
- 필드·메소드 사용
  - 클래스 내부는 이름만, 외부는 참조변수.이름
- t2 = t1
  - 객체가 아니라 번지가 복사됨 → 같은 객체를 공유
- 매개변수 전달
  - 기본 타입은 값이 복사, 객체는 번지가 복사
- 메소드 오버로딩
  - 이름은 같게, 매개변수의 타입·개수·순서는 다르게 (리턴 타입 무관)

수고하셨습니다.